

Data Collection (Epic 1)

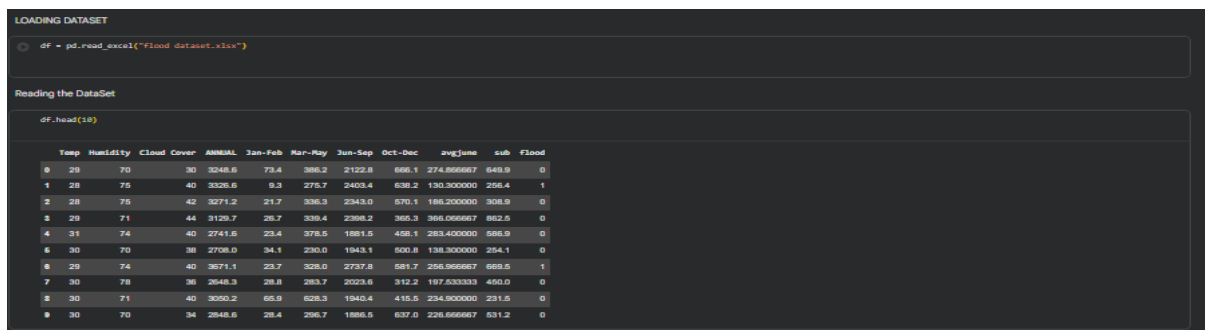
There are many popular open sources for collecting image datasets, such as Kaggle, open research repositories, and publicly shared computer vision datasets. In this project, we used a labeled fruit freshness image dataset containing categories for fresh and spoiled fruits. The dataset was downloaded from an open-source platform and organized into training and testing folders for model development.

Link: *(Provide your dataset link here — e.g., Kaggle fruit freshness dataset)*

Loading Data (USN2)

Our dataset consists of image files organized into class folders rather than traditional tabular formats like .csv or .excel. These images are loaded using deep learning data loaders that automatically read images from directories and prepare them for training.

In TensorFlow/Keras, utilities such as directory-based image loaders or image generators are used to read the dataset. As a parameter, we provide the folder path containing labeled images, allowing the system to automatically assign class labels and prepare the data for preprocessing and training.



The screenshot shows a Jupyter Notebook interface. The top section, titled 'LOADING DATASET', contains the code `df = pd.read_excel("Flood dataset.xlsx")`. Below this, a section titled 'Reading the DataSet' displays the output of `df.head(10)`. The resulting table has 11 columns: Temp, Humidity, Cloud Cover, AMMIL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgJuna, sub, and flood. The first 10 rows of data are shown, with the 'flood' column containing binary values (0 or 1).

	Temp	Humidity	Cloud Cover	AMMIL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgJuna	sub	flood
0	29	70	30	3248.6	73.4	388.2	2122.8	686.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	670.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	662.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	456.1	283.400000	586.9	0
5	30	70	38	2708.0	34.1	230.0	1943.1	500.8	138.300000	254.1	0
6	29	74	40	3671.1	23.7	328.0	2737.8	581.7	256.966667	669.5	1
7	30	78	36	2648.3	28.8	283.7	3023.6	312.2	197.533333	450.0	0
8	30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.900000	231.5	0
9	30	70	34	2648.6	28.4	286.7	1886.5	637.0	226.666667	531.2	0

Data Preparation (Epic 2)

Handling Missing or Invalid Data (USN3)

In an image-based dataset, missing values do not appear as empty cells like in tabular data. Instead, they may occur as corrupted image files, unreadable formats, or incorrectly labeled folders.

To check dataset integrity, we verify that all image files can be successfully loaded and that folder labels are correct. Image loading utilities are used to scan directories and confirm that each file is valid.

After performing this verification, we observed that the dataset contained properly formatted and accessible images. Therefore, no missing or corrupted data required correction, and we proceeded directly to the preprocessing stage.

```
#checking null values
dataset.isnull().any()
```

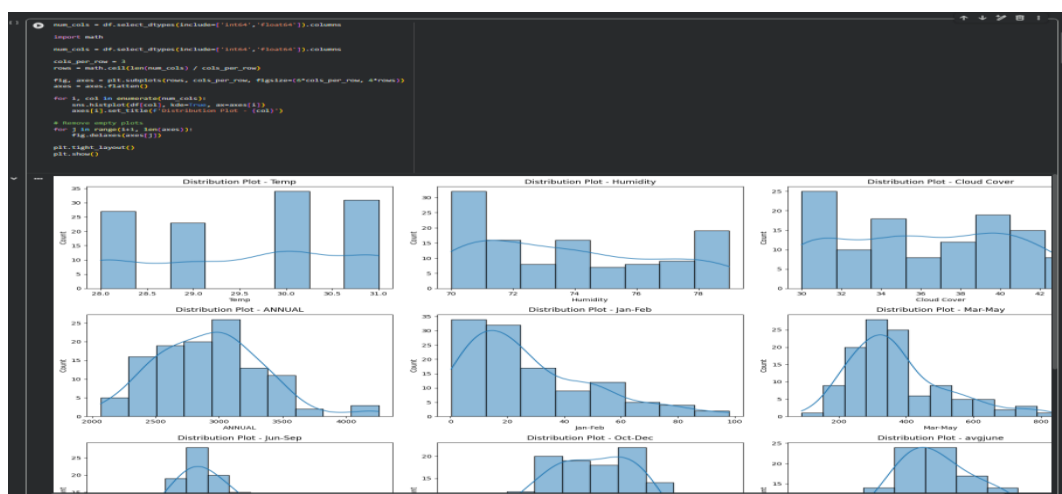
```
Temp      False
Humidity   False
Cloud Cover False
ANNUAL     False
Jan-Feb    False
Mar-May    False
Jun-Sep    False
Oct-Dec    False
avgjune    False
sub        False
flood      False
dtype: bool
```

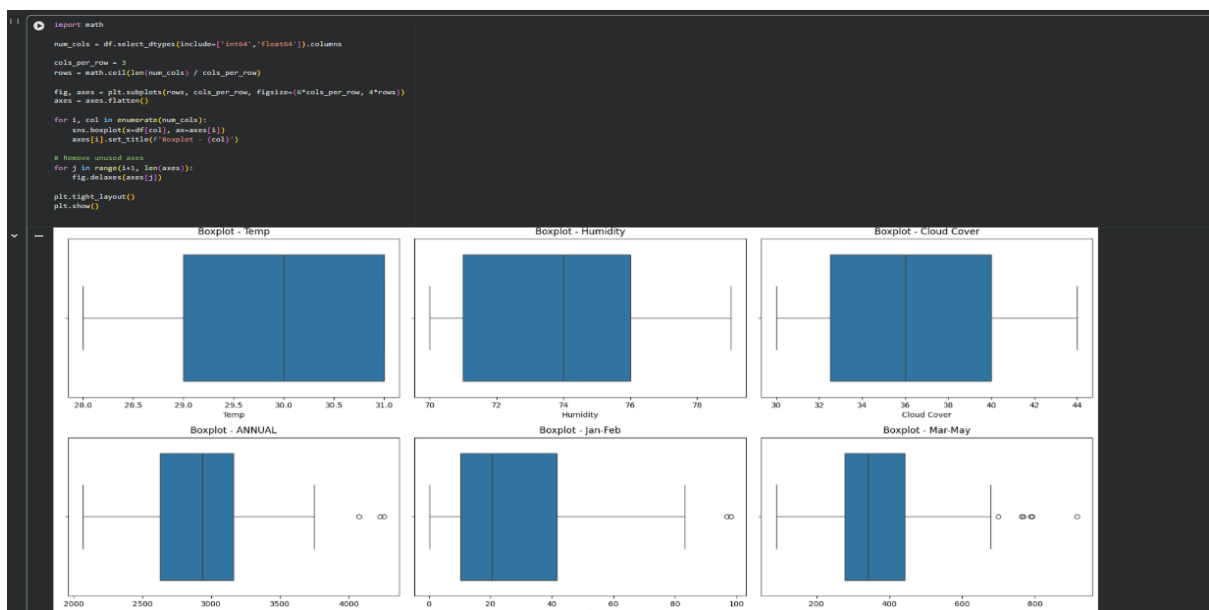
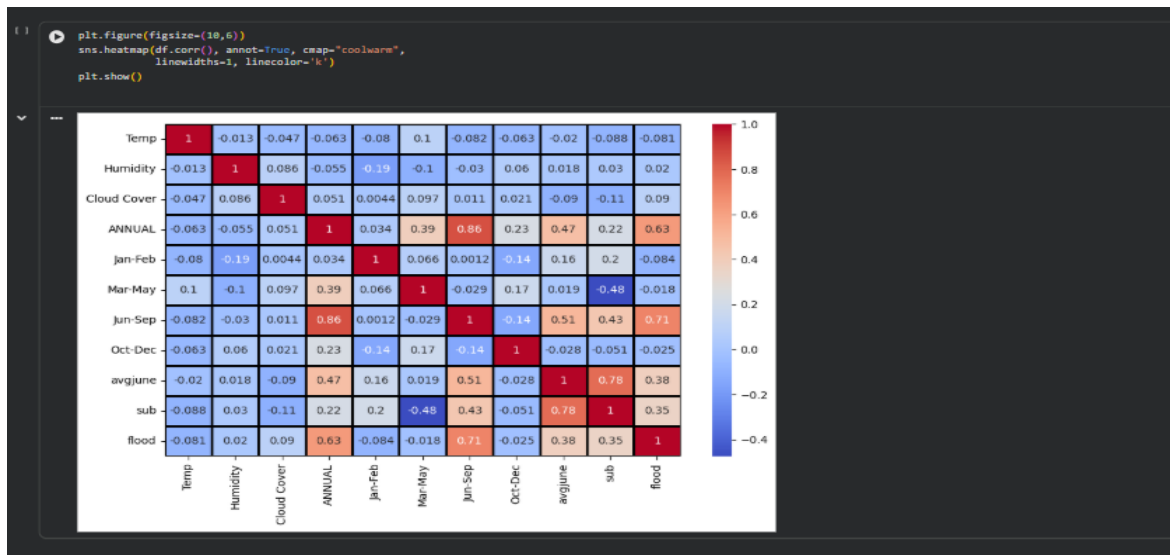
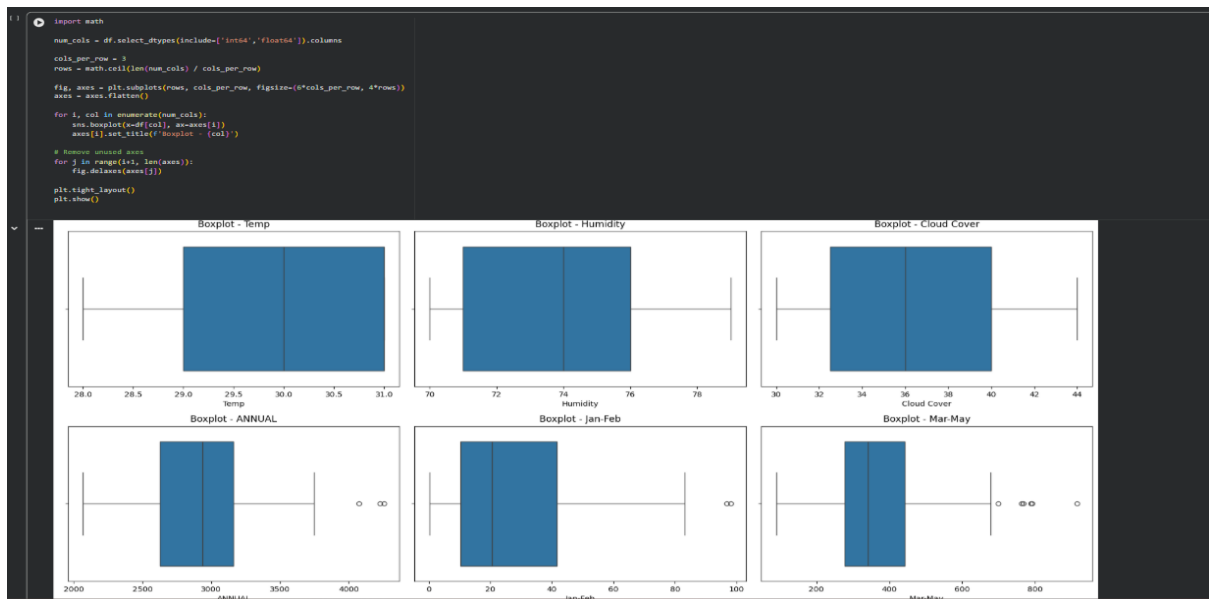
From the above code of analysis, we can infer that columns such as newbalanceOrg, oldbalnceOrig,isFraud are having the missing values, we need to treat them in a required way.

Total Story Point in Sprint 1= 2+1+3+3+3=12

Sprint 2

Data Visualization (Epic 3)

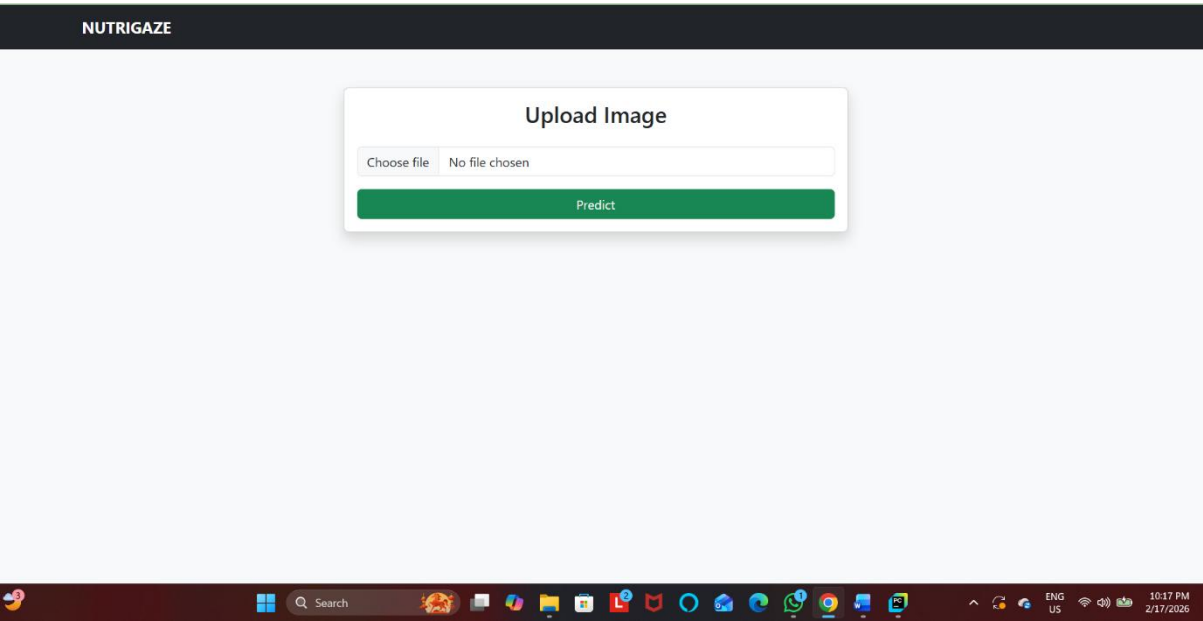




output (Epic 4)



Story (Epic 5)



Upload Image

Choose file No file chosen

Predict

Prediction

Banana_Rotten

Confidence: 100.0%



Search

